

E&M ANIMACIONES — API para Frontend React

Backend: Java 21 + Spring Boot 4.1.0

Puerto: `http://localhost:8080`

Auth: JWT Bearer Token

Pagina: `?page=0&size=10&sort=nombre,asc`

Fechas: ISO 8601 (`yyyy-MM-dd` para `LocalDate`, `HH:mm:ss` para `LocalTime`)

1. ARRANCAR EL BACKEND

```
# Requisitos: JDK 21, PostgreSQL en localhost:5432 con BD "em" creada
gradlew bootRun

# Variables de entorno opcionales:
# DB_USER=postgres      (default)
# DB_PASSWORD=postgres (default)
# JWT_SECRET=...        (default para dev)
```

El perfil `dev` (activo por defecto) usa `ddl-auto: update` — crea las tablas automaticamente.

2. CONEXION DESDE REACT

```
// .env
VITE_API_URL=http://localhost:8080/api

// src/api/client.ts
const API = axios.create({ baseURL: import.meta.env.VITE_API_URL });

// Interceptor para JWT
API.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Manejo de paginacion
interface Page<T> {
  content: T[];
  totalElements: number;
  totalPages: number;
  number: number; // pagina actual (0-index)
  size: number;   // tamaño de pagina
  first: boolean;
```

```
last: boolean;
}
```

3. ORDEN DE CONSTRUCCION DE PANTALLAS

El frontend debe seguir este orden porque hay dependencias (no podes crear evento sin cliente, no podes crear contrato sin evento + paquete, etc):

```
PASO 0: Login / Register
PASO 1: Categorias y Tematicas (dropdowns)
PASO 2: Clientes (necesarios para eventos)
PASO 3: Inventario (necesario para paquetes)
PASO 4: Paquetes (necesario para contratos)
PASO 5: CRONOGRAMA / Eventos ← EMPEZAR ACA para la vista principal
PASO 6: Contratos
PASO 7: Pagos
PASO 8: Plantillas
PASO 9: Dashboard
PASO 10: Generar PDF
```

Pero podes construir las pantallas 2-4 rapido (son CRUD simples de 1 tabla) y luego el cronograma/eventos que es la vista principal.

4. PANTALLAS — ENDPOINTS EXACTOS

PANTALLA 0: LOGIN / REGISTER

Login

```
POST /api/auth/login
Body (JSON):
{
  "email": "admin@em.com",
  "password": "123456"
}

Response 200:
{
  "token": "eyJhbGciOi...",
  "email": "admin@em.com",
  "nombre": "Admin",
  "apellido": "Proveedor",
  "rol": "PROVEEDOR",
  "proveedorId": 1
}
```

- Guardar `token` en `localStorage`.
- Guardar `proveedorId` para usarlo en todas las pantallas.

Register (solo primera vez)

POST `/api/auth/register`

Body (JSON):

```
{
  "nombre": "Admin",
  "apellido": "Proveedor",
  "email": "admin@em.com",
  "password": "123456",
  "telefono": "999888777",
  "nombreEmpresa": "E&M ANIMACIONES",
  "ruc": "10725783901",
  "nombreGerente": "Naysha Morales",
  "direccion": "Av. Principal 123",
  "telefonoEmpresa": "999888777"
}
```

Response 201 → mismo `JwtResponse` que login

Perfil empresa (header/sidebar)

GET `/api/auth/me` → { `id`, `email`, `nombre`, `apellido`, `rol`, `proveedorId` }

GET `/api/auth/proveedor` → { `id`, `nombreEmpresa`, `ruc`, `nombreGerente`, `logoUrl`, `telefono`, `email` }

PUT `/api/auth/proveedor` → Body: { `nombreEmpresa`, `ruc`, `nombreGerente`, `direccion`, `telefono`, `email` }

PANTALLA 1: CATEGORIAS Y TEMATICAS (publicas, para dropdowns)

GET `/api/categorias`

Response 200:

```
[
  { "id": 1, "nombre": "Show Infantil", "descripcion": "Fiestas de cumpleaños para niños" },
  { "id": 2, "nombre": "Hora Loca", "descripcion": null }
]
```

GET `/api/tematicas?categoriaId=1`

Response 200:

```
[
  { "id": 5, "nombre": "Mario Bross", "imagenReferencial": null, "categoria": { "id": 1, "nombre": "Show Infantil" } },
  { "id": 6, "nombre": "Frozen", "imagenReferencial": null, "categoria": { "id": 1, "nombre": "Show Infantil" } }
]
```

GET `/api/tematicas` → todas

GET `/api/tematicas/{id}` → una

GET `/api/categorias/{id}` → una

GET `/api/categorias/{id}/tematicas` → tematicas de esa categoria

PANTALLA 2: CLIENTES (CRUD)

Lista de clientes del proveedor. Necesarios para crear eventos (cada evento asigna un cliente).

```
GET /api/clientes?page=0&size=10&sort=nombreCompleto,asc
GET /api/clientes?q=Maria → buscar por nombre
GET /api/clientes?dni=12345678 → buscar por DNI exacto
GET /api/clientes/{id}
POST /api/clientes
PUT /api/clientes/{id}
DELETE /api/clientes/{id} → 204 No Content
```

Body POST/PUT:

```
{
  "nombreCompleto": "Iris Giovanna Reyes Herrada",
  "dni": "07257839",
  "telefono": "993405103",
  "direccion": "Pasaje 1 Mz D lote 16 - Urb. Los Girasoles",
  "referencia": "Frente al parque",
  "email": "iris@gmail.com"
}
```

Response:

```
{
  "id": 1,
  "nombreCompleto": "Iris Giovanna Reyes Herrada",
  "dni": "07257839",
  "telefono": "993405103",
  "direccion": "Pasaje 1 Mz D lote 16 - Urb. Los Girasoles",
  "referencia": "Frente al parque",
  "email": "iris@gmail.com",
  "fechaRegistro": "2026-07-25T14:30:00"
}
```

PANTALLA 3: INVENTARIO (CRUD con autocomplete)

```
GET /api/inventario?page=0&size=10
GET /api/inventario/search?q=muñeco → autocomplete ILIKE (devuelve List, no Page)
GET /api/inventario/{id}
POST /api/inventario
PUT /api/inventario/{id}
PATCH /api/inventario/{id}/deactivate → soft-delete (204)
DELETE /api/inventario/{id} → hard-delete (204)
```

Body POST/PUT:

```
{
  "nombre": "Muñeco Mario",
  "descripcion": "Muñeco inflable de Mario Bross 2m",
  "cantidadDisponible": 1,
}
```

```
"precioReferencial": 80.00
```

```
}
```

Response:

```
{
  "id": 3,
  "nombre": "Muñeco Mario",
  "descripcion": "Muñeco inflable de Mario Bross 2m",
  "cantidadDisponible": 1,
  "precioReferencial": 80.00,
  "estado": "ACTIVO"
}
```

PANTALLA 4: PAQUETES (CRUD con selector de items del inventario)

```
GET    /api/paquetes?page=0&size=10
GET    /api/paquetes?categoriaId=1      → filtrar por categoria
GET    /api/paquetes/{id}              → incluye array "detalles"
POST   /api/paquetes
PUT    /api/paquetes/{id}
PATCH /api/paquetes/{id}/deactivate
```

Body POST:

```
{
  "nombre": "PAQUETE PREMIUM",
  "descripcion": "Show completo con animadora, muñeco y DJ",
  "categoriaId": 1,
  "tematicaId": 5,
  "precioBase": 525.00,
  "duracionBaseHoras": 2.0,
  "detalles": [
    { "inventarioId": 1, "cantidadIncluida": 1, "precioUnitario": 50.00, "esObsequio": false,
    { "inventarioId": 3, "cantidadIncluida": 1, "precioUnitario": 80.00, "esObsequio": false,
    { "inventarioId": 10, "cantidadIncluida": 1, "precioUnitario": 0.00, "esObsequio": true,
  ]
}
```

Response:

```
{
  "id": 1,
  "nombre": "PAQUETE PREMIUM",
  "descripcion": "Show completo con animadora, muñeco y DJ",
  "precioBase": 525.00,
  "duracionBaseHoras": 2.0,
  "estado": "ACTIVO",
  "categoriaId": 1,
  "categoriaNombre": "Show Infantil",
  "tematicaId": 5,
  "tematicaNombre": "Mario Bross",
  "detalles": [
    { "id": 10, "inventarioId": 1, "inventarioNombre": "Animadora", "cantidadIncluida": 1, "p
```

```
{ "id": 11, "inventarioId": 3, "inventarioNombre": "Muñeco Mario", "cantidadIncluida": 1,
  { "id": 12, "inventarioId": 10, "inventarioNombre": "Tarjeta virtual", "cantidadIncluida": 1 }
}
```

PANTALLA 5: EVENTOS / CRONOGRAMA ★ EMPEZAR ACA ★

Esta es la pantalla principal. Muestra un calendario mensual con los eventos.

Calendario mensual

```
GET /api/eventos/calendario?inicio=2026-08-01&fin=2026-08-31&page=0&size=50
```

Responde con `Page<EventoResponse>`. Cada item tiene `fechaEvento`, `horaInicio`, `colorCalendario`, `estado`.

Crear evento (reservar fecha)

```
POST /api/eventos
Body:
{
  "clienteId": 1,
  "categoriaId": 1,
  "tematicaId": 5,
  "tipoEvento": "SHOW INFANTIL",
  "nombreCumpleanero": "Lucca Matías",
  "edadCumpleanero": 4,
  "fechaEvento": "2026-08-15",
  "horaInicio": "17:00:00",
  "horaFinEstimada": "19:00:00",
  "direccion": "Pasaje 1 Mz D lote 16 - Urb. Los Girasoles",
  "referencia": "Casa de rejas blancas",
  "aforoEstimado": 30,
  "colorCalendario": "#3B82F6",
  "notasInternas": "Cliente pidio tematica Mario Bross"
}
```

Estados del evento

```
PATCH /api/eventos/{id}/estado?estado=CONFIRMADO
```

Valores validos: `PROGRAMADO`, `CONFIRMADO`, `COMPLETADO`, `CANCELADO`

Otros endpoints

```
GET      /api/eventos/{id}
PUT      /api/eventos/{id}          → mismo body que POST
GET      /api/eventos/cliente/{clienteId}
DELETE   /api/eventos/{id}
```

PANTALLA 6: CONTRATOS

Cada evento tiene UN contrato (1:1). Se crea desde un paquete y los items se copian automaticamente.

```
POST      /api/contratos
Body:
{
  "eventoId": 1,
  "paqueteId": 1,
  "costoMovilidad": 50.00,
  "montoAdelanto": 100.00,
  "duracion": "2 horas de show + 30 min caritas pintadas",
  "observaciones": "Cliente frecuente"
}

GET       /api/contratos?page=0&size=10
GET       /api/contratos/{id}           → incluye array "detalles"
GET       /api/contratos/evento/{eventoId} → el contrato de un evento especifico
POST      /api/contratos/{contratoId}/detalles → agregar item manual
DELETE    /api/contratos/detalles/{detalleId} → quitar item
PATCH    /api/contratos/{id}/estado?estado=CONFIRMADO
```

Estados del contrato: BORRADOR , PENDIENTE , CONFIRMADO , COMPLETADO , CANCELADO

El `montoPendiente` se calcula automaticamente: `montoTotal - montoAdelanto`

PANTALLA 7: PAGOS

```
POST /api/pagos    (multipart/form-data)
Form fields:
  contratoId: 1
  tipoPago: ADELANTO      → valores: ADELANTO, SALDO, PAGO_TOTAL
  monto: 100.00
  metodoPago: YAPE        → valores: YAPE, PLIN, TRANSFERENCIA, EFECTIVO
  codigoOperacion: 987654321
  notas: Pago inicial
  comprobante: [archivo]  → opcional, JPG/PNG/PDF max 5MB

GET    /api/pagos/{id}
GET    /api/pagos/contrato/{contratoId}?page=0&size=10
GET    /api/pagos/pendientes?page=0&size=10
```

```
PATCH /api/pagos/{id}/verificar
PATCH /api/pagos/{id}/rechazar?motivo=Foto borrosa
```

PANTALLA 8: PLANTILLAS HTML

```
GET /api/plantillas?page=0&size=10
GET /api/plantillas/{id}
POST /api/plantillas
PUT /api/plantillas/{id}
PATCH /api/plantillas/{id}/deactivate
DELETE /api/plantillas/{id}

Body POST:
{
  "nombre": "Contrato Show Infantil",
  "descripcion": "Plantilla base para shows infantiles",
  "tipo": "CONTRATO",
  "contenidoHtml": "<html><body><h1>{{PROVEEDOR_NOMBRE}}</h1>...</body></html>",
  "placeholders": "{{PROVEEDOR_NOMBRE}}, {{CLIENTE_NOMBRE}}, ...",
  "esDefault": true
}
```

Los 23 placeholders disponibles estan en el PDF generado (ver abajo).

PANTALLA 9: DASHBOARD

```
GET /api/dashboard

Response:
{
  "totalClientes": 45,
  "totalInventarioActivo": 120,
  "totalPaquetesActivos": 8,
  "totalEventosProgramados": 12,
  "totalContratos": 30,
  "contratosPorEstado": {
    "BORRADOR": 5, "PENDIENTE": 10, "CONFIRMADO": 8,
    "COMPLETADO": 5, "CANCELADO": 2
  },
  "ingresosTotales": 15000.00,
  "pagosPendientes": 3,
  "montoPendienteCobrar": 5000.00,
  "proximosEventos": [ ...30 dias, incluye fechaEvento, horaInicio, clienteNombre, tematicaNo
  "eventosDelMes": [ ...mes actual ]
}
```

PANTALLA 10: GENERAR PDF DEL CONTRATO

POST /api/documentos/generar/{contratoId}

Response 201:

```
{
  "id": 5,
  "contratoId": 1,
  "plantillaId": 1,
  "urlPdf": "/uploads/pdfs/contrato_1.pdf",
  "version": 1,
  "fechaGeneracion": "2026-07-25T14:40:00"
}
```

GET /api/documentos/contrato/{contratoId} → historial de versiones

Los placeholders que se reemplazan en el HTML:

Placeholder	Fuente
{{PROVEEDOR_NOMBRE}}, {{PROVEEDOR_RUC}}, {{PROVEEDOR_GERENTE}}	Tabla proveedor
{{CLIENTE_NOMBRE}}, {{CLIENTE_DNI}}, {{CLIENTE_TELEFONO}}, {{CLIENTE_DIRECCION}}, {{CLIENTE_REFERENCIA}}	Tabla cliente
{{EVENTO_TIPO}}, {{EVENTO_TEMATICA}}, {{EVENTO_FECHA}}, {{EVENTO_HORA_INICIO}}, {{EVENTO_HORA_FIN}}, {{EVENTO_NOMBRE_CUMPLEANERO}}, {{EVENTO_EDAD_CUMPLEANERO}}	Tabla evento
{{CONTRATO_MONTO_TOTAL}}, {{CONTRATO_MOVILIDAD}}, {{CONTRATO_MONTO_ADELANTO}}, {{CONTRATO_MONTO_PENDIENTE}}, {{CONTRATO_DURACION}}	Tabla contrato
{{CONTRATO_DETALLE_ITEMS}}	Items del contrato NO obsequio (tabla HTML)
{{CONTRATO_OBSEQUIOS}}	Items marcados como obsequio (tabla HTML)
{{CONTRATO_TERMINOS}}	proveedor.terminosCondiciones
{{FECHA_EMISION}}	Fecha actual

5. ERRORES — FORMATO DE RESPUESTA

Todos los errores devuelven:

```
{
  "timestamp": "2026-07-25T14:30:00",
  "status": 404,
  "error": "Not Found",
  "message": "Cliente no encontrado con id: 99"
}
```

Errores de validacion:

```
{
  "timestamp": "2026-07-25T14:30:00",
  "status": 400,
  "errors": {
```

```
"nombreCompleto": "no debe estar vacio",  
"dni": "no debe estar vacio"  
}  
}
```

401 → JWT invalido o expirado (redirigir a login) 403 → Token valido pero no tiene rol PROVEEDOR

6. FLUJO COMPLETO DE PANTALLAS

```
[Login] → [Dashboard]  
    └─ [Clientes] → CRUD rapido  
    └─ [Inventario] → CRUD + autocomplete  
    └─ [Paquetes] → elegir categoria, tematica, items del inventario  
    └─ [★ Cronograma ★] → calendario mensual, crear/editar eventos  
        └─ click en evento → [Evento detalle]  
            └─ [Contrato] → crear desde paquete  
                └─ [Pagos] → registrar pago, subir voucher  
                └─ [Generar PDF]  
    └─ [Plantillas] → CRUD de templates HTML  
    └─ [Perfil] → datos de la empresa
```

7. CHECKLIST DE LO QUE NECESITA EL FRONTEND

- [] Pantalla Login/Register
- [] Guardar JWT en localStorage
- [] Axios interceptor para Bearer token
- [] Componente Page<T> para manejar paginacion
- [] Componente de calendario (FullCalendar o similar)
- [] Selects de Categoria/Tematica (fetch al montar)
- [] CRUD Clientes (formulario + tabla con paginacion)
- [] CRUD Inventario (formulario + tabla + search bar con autocomplete)
- [] CRUD Paquetes (formulario con multi-select de items del inventario)
- [] Calendario/Eventos: fetch `GET /api/eventos/calendario` con rango del mes visible
- [] Modal/Formulario para crear evento (necesita clienteld y categoriald de dropdowns)
- [] Cambiar estado del evento (PROGRAMADO → CONFIRMADO → COMPLETADO)
- [] Vista de contrato (crear desde paquete, ver items, agregar/quitar)
- [] Registro de pago con upload de comprobante
- [] Dashboard con KPIs
- [] Generar PDF (llamar POST y mostrar link/boton descarga)